بسمه تعالی

تمرین Git

مهندسی اینترنت

پاییز ۱۴۰۵

مهلت تحویل: ۲۶ مهر ۱۴۰۵

فهرست مطالب

۱ مقدمه ۲

۲ سؤال‌های مفهومی ۲

۱.۲ سؤال یک: merge و rebase ۲

۲.۲ سؤال دو: انتخاب بخشی از تغییرات ۳

۳.۲ سؤال سوم: یافتن منشأ خطا ۳

۴.۲ سؤال چهارم: بررسی تغییرات پیش از کامیت ۳

۵.۲ سؤال پنجم: رفع merge conflict در کار گروهی ۳

۶.۲ سؤال ششم: انتخاب روش مناسب برای بازگرداندن تغییرات ۴

۳ نحوهٔ تحویل ۴

۴ قوانین ۴

۱ مقدمه

هدف این تمرین آشنایی با مفاهیم و تصمیم‌گیری در موقعیت‌های رایج کار با Git است. همهٔ سؤال‌ها به‌صورت نوشتاری پاسخ داده می‌شوند. در سؤال‌های سناریومحور، مراحل پیشنهادی، دستورهای لازم و دلیل انتخاب روش خود را توضیح دهید؛ اجرای دستورها، ساخت مخزن یا ارائهٔ تصویر و خروجی واقعی لازم نیست. مجموع نمرهٔ تمرین ۲۰ است.

۲ سؤال‌های مفهومی

۱.۲ سؤال یک: merge و rebase

۳ نمره

دستورهای git merge و git rebase چه کاری انجام می‌دهند؟ تفاوت آن‌ها را از نظر نحوهٔ ترکیب تغییرات و شکل تاریخچهٔ کامیت‌ها توضیح دهید. برای هر کدام یک موقعیت مناسب استفاده بیان کنید و توضیح دهید چرا در آن موقعیت آن روش را انتخاب می‌کنید.

۲.۲ سؤال دو: انتخاب بخشی از تغییرات

۳ نمره

منظور از بخش تغییرات (hunk) در خروجی مقایسهٔ تغییرات (diff) در Git چیست؟ توضیح دهید چگونه می‌توان با استفاده از خط فرمان، تغییرات یک فایل را به‌صورت تعاملی بررسی کرد و فقط بعضی از بخش‌ها را برای کامیت بعدی در ناحیهٔ staging قرار داد. دستورها و انتخاب‌های لازم را ذکر کنید و بگویید این کار چه فایده‌ای برای ساختن کامیت‌های منسجم دارد.

۳.۲ سؤال سوم: یافتن منشأ خطا

۳ نمره

دستور git bisect چگونه کار می‌کند؟ مراحل پیدا کردن کامیتی را که یک خطا را وارد کرده است، از مشخص کردن یک کامیت سالم و یک کامیت معیوب تا پایان جست‌وجو، توضیح دهید. یک مثال از کاربرد آن بیاورید و توضیح دهید چرا این روش برای یافتن منشأ خطا مفید است.

۴.۲ سؤال چهارم: بررسی تغییرات پیش از کامیت

۳ نمره

فرض کنید فایل README.md در مخزن tracked است و قبلاً کامیت شده است. بخشی از تغییرات جدید آن را وارد ناحیهٔ staging کرده‌اید و سپس، بدون اجرای دوبارهٔ دستور افزودن به این ناحیه، همان فایل را مجدداً ویرایش کرده‌اید. همچنین پوشه‌ای به نام tmp/ دارید که فقط شامل فایل‌های موقت و untracked است.

۱. با چه دستورهایی وضعیت مخزن، تغییرات هنوز واردنشده به ناحیهٔ staging و تغییرات آمادهٔ کامیت را بررسی می‌کنید؟ تفاوت دو دستهٔ تغییرات اخیر را توضیح دهید.

۲. اگر در همین وضعیت یک کامیت معمولی و بدون گزینهٔ a- ثبت کنید، کدام تغییرات فایل در آن قرار می‌گیرند و کدام تغییرات باقی می‌مانند؟ دلیل پاسخ خود را بیان کنید.

۳. برای جلوگیری از اضافه‌شدن فایل‌های پوشهٔ tmp/ به کامیت‌های بعدی، چه تنظیمی و در کدام فایل انجام می‌دهید؟ آیا همین تنظیم برای فایلی که قبلاً tracked شده است نیز کافی است؟ توضیح دهید.

۵.۲ سؤال پنجم: رفع merge conflict در کار گروهی

۴ نمره

شما و هم‌تیمی‌تان از یک کامیت مشترک شروع کرده‌اید. شما در branch feature-description و هم‌تیمی‌تان در branch main، یک خط از فایل README.md را به دو متن متفاوت تغییر داده و کامیت کرده‌اید. همهٔ تغییرات ثبت شده‌اند و working tree در وضعیت clean است؛ یعنی تغییر ثبت‌نشده‌ای ندارید. هنگام merge کردن branch شما در main، Git برای این خط conflict گزارش می‌کند.

۱. چرا این conflict رخ داده است؟ آیا هر تغییری که دو نفر در یک فایل ایجاد کنند الزاماً باعث conflict می‌شود؟

۲. چگونه فایل‌های دارای conflict و دو نسخهٔ متن را شناسایی می‌کنید؟ نقش conflict markers در فایل را توضیح دهید.

۳. فرض کنید اطلاعات هر دو تغییر باید حفظ شود. از زمان مشاهدهٔ conflict تا تکمیل merge چه مراحلی را طی می‌کنید و چه دستورهایی به کار می‌برید؟ چگونه مطمئن می‌شوید که conflict markers در متن نهایی باقی نمانده‌اند؟

۴. اگر هنگام بررسی متوجه شوید که فعلاً نباید این merge انجام شود، با چه دستوری عملیات merge در حال انجام را لغو می‌کنید؟

۶.۲ سؤال ششم: انتخاب روش مناسب برای بازگرداندن تغییرات

۴ نمره

در هر یک از موقعیت‌های مستقل زیر، توضیح دهید چه می‌کنید، چه دستورهایی به کار می‌برید و چه اثری بر working tree، ناحیهٔ staging یا تاریخچهٔ کامیت‌ها می‌گذارند. فرض کنید هنگام اجرای دستورهای بازگردانی conflict رخ نمی‌دهد.

۱. در فایل tracked به نام notes.txt تغییری ایجاد کرده‌اید که دیگر آن را نمی‌خواهید. این تغییر وارد ناحیهٔ staging نشده است و نسخهٔ موجود در این ناحیه با آخرین کامیت یکسان است. می‌خواهید فقط تغییرات این فایل کنار گذاشته شوند و فایل‌های دیگر دست‌نخورده بمانند.

۲. تغییرات notes.txt را اشتباهاً وارد ناحیهٔ staging کرده‌اید، اما هنوز کامیت نکرده‌اید. می‌خواهید فایل از این ناحیه خارج شود، ولی ویرایش‌های شما در working tree حفظ شوند.

۳. یک کامیت اشتباه معمولی، نه merge commit، را روی branch مشترک main، push کرده‌اید و هم‌تیمی‌ها نیز آن را دریافت کرده‌اند. working tree در وضعیت clean است. می‌خواهید اثر این کامیت را بدون حذف یا بازنویسی تاریخچه بازگردانید. روش مناسب چیست و چرا حذف کامیت از تاریخچه و force push در این موقعیت انتخاب مناسبی نیست؟

۳ نحوهٔ تحویل

پاسخ هر شش سؤال را در فایل README.md مخزن خود، با شمارهٔ سؤال مربوط، بنویسید. در پاسخ‌ها صرفاً نام دستور را ننویسید؛ گزینه‌ها و آرگومان‌های لازم، دلیل انتخاب و نتیجهٔ مورد انتظار را نیز توضیح دهید. برای نام شاخه، مسیر فایل یا شناسهٔ کامیت می‌توانید از مقادیر فرضی مشخص استفاده کنید.

۴ قوانین

- **تحویل.** کار خود را در مخزن گیت‌هاب درس تحویل دهید. تحویل از راه دیگر (ایمیل، پیام‌رسان) پذیرفته نیست، مگر آنکه پیشاپیش هماهنگ شده باشد.
- **مهلت.** ۲۶ مهر ۱۴۰۵. آخرین وضعیت مخزن پیش از پایان مهلت ارزیابی می‌شود.
- **تأخیر.** تا ۷۲ ساعت پس از مهلت، با کسر ۲۰ درصد از نمره‌ی تمرین پذیرفته می‌شود؛ پس از آن پذیرفته نیست.
- **اصالت کار.** گفت‌وگو درباره‌ی صورت مساله آزاد است، اما پاسخی که تحویل می‌دهید باید نوشته‌ی خودتان باشد. تحویل پاسخ مشترک، برای **هر دو** طرف نمره‌ی صفر دارد.

- **ابزارهای هوش مصنوعی.** استفاده از آن‌ها مجاز است، به سه شرط: در فایل README بنویسید کجا و برای چه از آن‌ها استفاده کرده‌اید، بتوانید هر بخش از پاسخی که تحویل می‌دهید را توضیح دهید، و بدانید که در جلسه‌ی تحویل حضوری پرسیده می‌شود. پاسخی که نتوانید توضیح دهید، تحویل داده نشده به شمار می‌آید.
- **نمره‌دهی.** بارم هر بخش کنار همان بخش آمده است. درستی پاسخ شرط لازم است، اما تمام نمره نیست: دلیل انتخاب‌ها و روشنی توضیح هم دیده می‌شوند.